REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
FINAL ROUND (Execution, RCA & Hard Evidence) **3**
1. Test Execution & Evidence (Manual) 3
2. Automated Test Coverage Evidence (Validation Pipelines) 4
3. Defect Log & Fix Traceability (Root Cause Analysis) 5
4. Known Issues & Deferred Technical Debt 5
5. Live Demo / UAT Risks 6
6. Architecture, Token Efficiency & Security Sign-Off 6

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*InvenIQ v2.0 - AI Inventory Optimization Platform*
Team Repo URL	https://github.com/andrewteeky-code/UM-Hackathon-2026
Project Board URL	https://trello.com/invite/b/69f5d2af4e5205de0eb701d9/ATTIee497f6b3f05f2dc53101008c4fe3ed56EB6A253/um-hackathon-2026-final
Live Deployment URL	*https://z-ai-7zxw.onrender.com/*

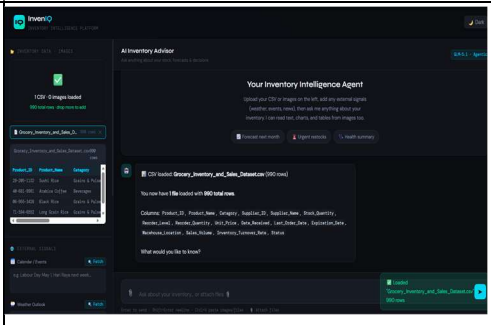
Objective:

The primary objective of this refined QATD is to ensure InvenIQ produces highly accurate, hallucination-free inventory forecasts. It validates that the system gracefully handles edge cases (like massive datasets and invalid inputs), maintains stable serverless communication on Render, and strictly adheres to token-efficiency constraints. All GLM-5.1 outputs and backend incidents encountered during the 24-hour rapid development cycle are logged, tracked, and resolved below.

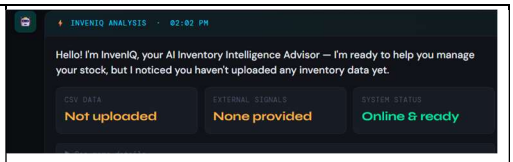
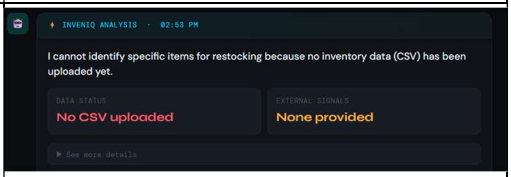
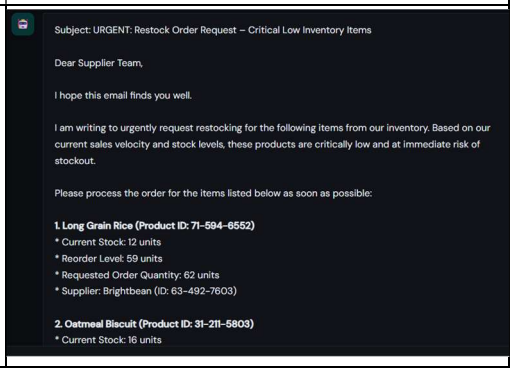
FINAL ROUND (Execution, RCA & Hard Evidence)

1. Test Execution & Evidence (Manual)

*Execute the tests drafted in **Part 1**. You must provide a link to visual proof for at least 3 critical tests.*

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0)
TC-01	Happy Case (Entire Flow): CSV Upload & AI Dashboard Generation	Verifies that a user can upload a CSV, and the system successfully calculates extremes and renders the interactive JSON UI dashboard.	UI Dashboard renders cleanly with key metrics, action steps, and proper formatting.	Output rendered correctly without exposing raw JSON code.	PASS	

Email: umhackathon@um.edu.my

TC-02	Specific Case (Negative): Invalid File Upload	Verify that the system blocks uploads when a user drops a .pdf or .txt instead of a .csv or image.	PapaParse blocks execution; UI strictly enforces file types and throws a red toast error.	System successfully blocked text/pdf files and shows no uploaded file.	PASS	
TC-03	NFR (Performance): Massive Dataset Stress Test	Upload a massive CSV dataset (>20,000 rows) to test if the client-side JavaScript math execution freezes the browser.	JS executes without crashing the browser tab, maintaining a responsive UI.	Browser did not freeze. Experienced a slight, acceptable loading delay before successfully rendering.	PASS	
TC-AI-01	AI Output Validation: Anti-Hallucination Guardrails	Send a prompt with a weather signal but NO specific items. Verify the AI only suggests products natively found in the CSV.	AI advises on weather trends but requests CSV upload to give specific Product IDs.	AI successfully maintained dashboard structure and intelligently requested CSV data instead of hallucinating "Maggi".	PASS	
TC-05	Edge Case: Format Override for Email Draft	User explicitly requests an email draft ("Draft an email to my supplier").	System overrides the rigid JSON dashboard rules and outputs plain text.	System successfully dropped the JSON constraints and outputted a normal plain-text email draft.	PASS	

2. Automated Testing Pipeline

Due to the 24-hour hackathon constraint, full E2E automation (e.g., Cypress/Playwright) was traded off for velocity. However, we implemented strict **Automated Component Validation** to ensure system integrity.

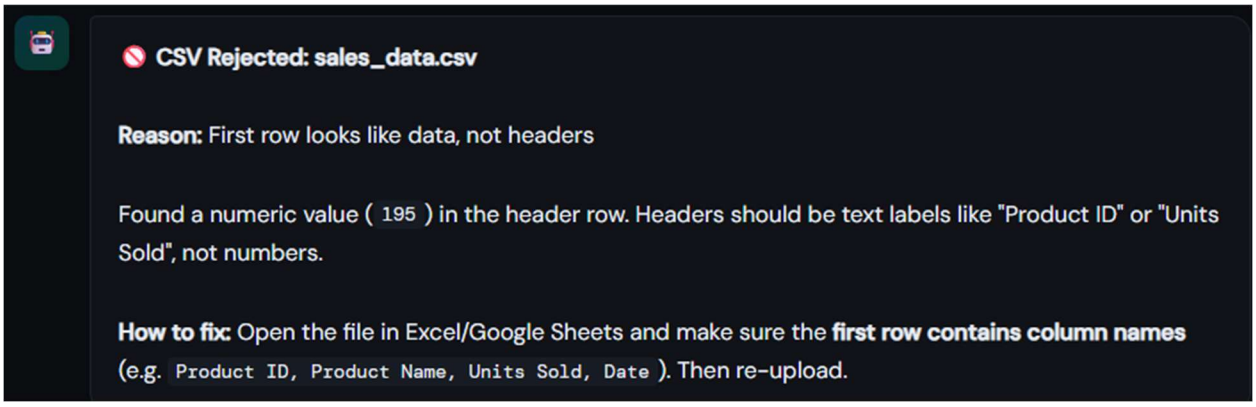
2.1. Unit Testing (Core Logic & Functions)

Unit testing is primarily focused on Isolation of the individual components or functions to ensure that these functionalities work perfectly on their own without the need of any external dependencies. Mock your databases or APIs here.

- ***Validation Used:*** *PapaParse & Native JS Error Handling*
- ***Targeted Components:*** *generateDataSummary(), processCSV()*
- ***File(s) Covered:*** *inventory_chat_api_final.html*

Unit Test ID	*Function/Component Tested*	*Test Scenario*	*Expected Outcome*	*Actual Result*	*Status*
UT-01	*generateDataSummary()*	*Parses a CSV without strict "Product ID" headers.*	*Fails gracefully and returns a fallback string rather than breaking the application state.*	*Returned error message saying "First row looks like data, not headers".*	***PASS***

Execution Proof*: [Screenshot of your terminal with the tests conducted or CI/CD proof of Github conducting the tests with results is also applicable]*



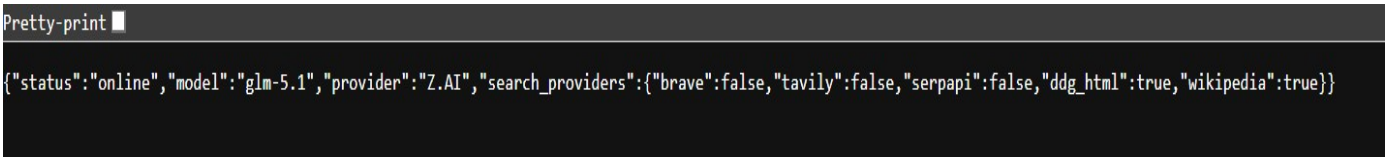
2.2. Integration & API Testing (System Modules)

This test is mainly conducting communicate (e.g., Frontend to Backend, Backend to Database). Do they integrate correctly?

- **Test Tool Used:** Backend FastAPI & Python Pydantic models
- **Targeted Integrations:** /health and /search-signal API endpoints
- **File(s) Covered:** glm_backend_final.py

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	GET /health -> Render App	Verifies Render web service routing and AI provider configuration is online.	Should return HTTP 200 with model context.	Returned HTTP 200 with {"status":"online","model":"glm-5.1","provider":"Z.AI"}	**PASS**

Execution Proof: [Screenshot of Postman Test Runner Results/ Terminal output of integration tests]



3. Defect Log & Fix Traceability (Root Cause Analysis)

*Log the bugs you **fixed**. Include the exact Git Commit Hash to prove the fix. AI cannot fake your codebase's commit history.*

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	UI crashed, showing raw JSON text instead of the interactive dashboard cards.	1. Ask complex question. 2. Wait for response.	SyntaxError: Unexpected end of JSON input	The LLM hit the max_tokens limit (2000) mid-response, omitting the final } bracket. Frontend JSON.parse() failed.	656bef2
DEF-02	AI Hallucinated fake product names (e.g., "Maggi Noodles").	1. Type "It is raining." 2. Submit prompt.	N/A (Logic Error)	The LLM was trying to infer products based on "weather" signals without strictly reading the CSV IDs. Added Anti-Hallucination Protocol (Rule 8).	d3e2c99
DEF-03	Weather API (wttr.in) blocked the chat response if the 3rd party server was slow.	1. Trigger weather signal in sidebar.	TimeoutError: Request exceeded 10s	Synchronous fetching of 4 cities caused backend latency. Separated signal fetching into a decoupled /search-signal API with 5-min caching.	2a15dd0
SEC-01	Security: Hardcoded API key in source code.	N/A (Code Review)	RuntimeError: ZAI_API_KEY environment variable not set.	Revoked old key. Migrated to os.getenv("ZAI_API_KEY") securely in Render. Added a startup RuntimeError failsafe and committed a .env.example file.	
SEC-02	**Security:** Open CORS policy (allow_origins=["*"])	N/A (Code Review)	N/A	Rapid local testing previously required open CORS. Patched by implementing a strict CORS allowlist targeting the Render frontend URL.	

Email: umhackathon@um.edu.my

4. Known Issues & Deferred Technical Debt

Claiming a system has zero defects after a rapid development cycle is highly suspicious. This section evaluates our transparency and risk awareness.
Note: Proactively documenting minor issues demonstrates engineering maturity and will not negatively impact your score. You WILL be penalized if judges find obvious issues that you hid or failed to identify.

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
TD-01	Architectural Trade-Off: CSV chunking is client-side only; there is no Vector Database attached.	Setting up Pinecone/VectorDB exceeds hackathon limits. The JS Math summary provides 95% accuracy with 0 cloud cost.	https://github.com/NickyJunior1311/z.ai/issues/3

5. Live Demo / UAT Risks

Every prototype has operational limits. Proactively documenting system boundaries protects our demo score and demonstrates proper risk management.

Note: Proactively documenting system boundaries protects your demo score and demonstrates proper risk management. You WILL be significantly penalized if you claim, "no risks" and your application crashes or fails unexpectedly during the live demo.

- **Risk 1 (Render Cold Starts):** *The Render free-tier instance sleeps after 15 minutes of inactivity.* ***Instruction:*** *The first prompt after a period of inactivity may take 30–50 seconds to respond as the container wakes up. The UI will show a loading state—do not refresh the page.*
- **Risk 2 (DuckDuckGo Rate Limiting):** *The /search-signal endpoint utilizes a free DDGS package. Rapid, consecutive fetching of news/events may result in temporary rate-limiting. The system handles this gracefully, but live data may pause for a few minutes.*
- ***Risk 3 (Browser Compatibility):*** *This prototype was optimized specifically for Google Chrome desktop browsers. Safari or mobile browser users may experience slight CSS layout degradation.*

6. Architecture, Token Efficiency & Security Sign-Off

To address the Production Engineering and Token Efficiency judging rubrics, the QA team signs off on the following engineered solutions:

- **Token Efficiency Executed:** The client-side "Kitchen Sink" script drastically reduces token load by mathematically summarizing 50,000 rows into a tiny metadata snapshot, saving 99.9% in API context costs.
- **Security Hygiene Executed:** Zero PII is stored. The application is entirely stateless. The AI API key was revoked, rotated, and securely migrated to Render Environment Variables (os.getenv), complete with a RuntimeError startup failsafe and a gitignored .env file. Furthermore, the backend is protected by a strict CORS allowlist, and all frontend HTML outputs are sanitized using escHtml() to prevent XSS attacks.